

Guía Completa de Trabajo — Proyecto Final

Curso 1: Git & GitHub para Data Science

Instituto de Analítica y Negocios — Lima, Perú

Docente: Ing. Marco Mayta

Descripción General

Construirás un pipeline de análisis de datos cardiovasculares trabajando con Git desde el primer archivo. El análisis usa el Heart Disease Dataset de la UCI, que se descarga automáticamente con el notebook que recibes.

El foco no está en el análisis en sí, sino en cómo lo construyes: cada sección tiene su propio commit, el historial debe poder leerse como una historia y las ramas deben tener un propósito claro.

Archivos que recibes

- `main_ayuda.ipynb` — el notebook con todo el código del proyecto
- `guia.txt` o `guia.pdf` — este archivo con todas las instrucciones

No necesitas crear ningún archivo de código. Solo sigue las instrucciones en el orden en que aparecen.

Herramientas necesarias

- Git instalado y configurado con tu nombre y correo
- Python 3.9 o superior
- Jupyter Notebook o JupyterLab (o VS Code con extensión Jupyter)
- Una terminal (`cmd`, `PowerShell`, `bash` o la terminal de VS Code)

Antes de empezar, verifica que Git está configurado:

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu@correo.com"
```

Para verificarlo:

```
git config --global user.name
git config --global user.email
```

FASE 1 — Inicialización y commits por sección

El objetivo de esta fase es construir el pipeline de análisis paso a paso, haciendo un commit después de cada sección del notebook.

Paso 1 — Crear la carpeta del proyecto

Crea una carpeta nueva con el nombre que quieras para el proyecto:

```
mkdir proyecto_cardio
cd proyecto_cardio
```

Paso 2 — Inicializar el repositorio Git

Dentro de la carpeta del proyecto, ejecuta:

```
git init
```

Paso 3 — Crear la estructura de carpetas

Crea dos carpetas vacías dentro del proyecto:

```
mkdir data
mkdir outputs
```

Git no versiona carpetas vacías. Crea un archivo `.gitkeep` dentro de cada una:

Windows (cmd):

```
type nul > data\.gitkeep
type nul > outputs\.gitkeep
```

Linux / macOS / Git Bash:

```
touch data/.gitkeep
touch outputs/.gitkeep
```

Paso 4 — Crear el archivo `.gitignore`

En la raíz del proyecto crea un archivo llamado `.gitignore` con el siguiente contenido:

```
__pycache__/  
*.pyc  
*.pyo  
.env  
.DS_Store  
data/*.csv  
outputs/*.png  
outputs/*.csv
```

Paso 5 — Crear el archivo README.md

En la raíz del proyecto crea README.md con el siguiente contenido (reemplaza los datos entre corchetes):

```
# Pipeline de Análisis - Heart Disease Dataset

Análisis exploratorio del dataset cardiovascular de la UCI.
El pipeline carga, limpia y analiza 303 registros clínicos
para explorar patrones asociados a enfermedades cardíacas.

## Autor
[Tu nombre completo]
Instituto de Analítica y Negocios - Lima, Perú
Curso 1: Git & GitHub para Data Science

## Instalación
pip install ucimlrepo pandas matplotlib seaborn

## Ejecución
Abrir y ejecutar el notebook main.ipynb en orden.

## Estructura del proyecto
    data/          Datos generados por el notebook (ignorados por Git)
    outputs/       Figuras y resúmenes generados (ignorados por Git)
    main.ipynb     Notebook principal del pipeline
    README.md      Descripción del proyecto
    .gitignore     Archivos excluidos del control de versiones
```

Paso 6 — Primer commit: estructura del proyecto

```
git add .
git commit -m "chore: inicializar estructura del proyecto"
git log --oneline
```

Paso 7 — Mover el notebook al proyecto y hacer el segundo commit

Copia o mueve main_ayuda.ipynb a la carpeta del proyecto y crea tu propio main.ipynb. Luego:

```
git add main.ipynb
git commit -m "chore: agregar notebook principal del pipeline"
```

Paso 8 — Instalar las dependencias

```
pip install ucimlrepo pandas matplotlib seaborn
```

Paso 9 — Sección 1 del notebook: Carga del dataset

Copia la **Sección 1** del notebook y ejecútala. Luego:

```
git add main.ipynb
git commit -m "feat: cargar dataset cardiovascular desde UCI"
```

Paso 10 — Sección 2 del notebook: Inspección inicial

Copia la **Sección 2** y ejecútala. Luego:

```
git add main.ipynb
git commit -m "feat: inspeccion inicial del dataset"
```

Paso 11 — Sección 3 del notebook: Limpieza de datos

Copia la **Sección 3** y ejecútala. Luego:

```
git add main.ipynb
git commit -m "feat: limpieza de valores invalidos y duplicados"
```

Paso 12 — Sección 4 del notebook: Análisis exploratorio (EDA)

Copia la **Sección 4** y ejecútala. Luego:

```
git add main.ipynb
git commit -m "feat: analisis exploratorio con tres visualizaciones"
```

Paso 13 — Verificar el historial de la Fase 1

Ejecuta `git log --oneline` y verifica que aparezcan estos 6 commits en orden. Luego pega el resultado en el `README.md` bajo la sección `## Historial de la Fase 1` y haz:

```
git add README.md
git commit -m "docs: registrar historial de fase 1 en README"
```

FASE 2 — Corrección del historial

El objetivo es practicar `git reset` para deshacer un commit sin perder los cambios.

Paso 1 — Ejecutar la celda de la Fase 2

En el notebook, busca y copia la sección **FASE 2 — Corrección del historial** a tu `main.ipynb` y ejecútala. Verifica que el `README.md` fue modificado automáticamente.

Paso 2 — Hacer el commit con mensaje malo (a propósito)

```
git add .  
git commit -m "cambios"
```

Paso 3 — Deshacer el commit sin perder los cambios

```
git reset HEAD~1
```

Este comando deshace el último commit pero conserva los cambios en los archivos. Verifica:

```
git log --oneline  
git status
```

Paso 4 — Rehacerlo con un mensaje correcto

```
git add .  
git commit -m "docs: agregar resultado clave del EDA en README"
```

Paso 5 — Documentar la decisión en el README.md

Agrega al final del README.md la siguiente sección:

```
## Decisiones del historial
```

En la Fase 2 se usó ‘git reset HEAD~1’ para deshacer un commit cuyo mensaje no era descriptivo ("cambios"). Los archivos modificados se conservaron y se volvieron a commitear con un mensaje que explica claramente qué cambió y por qué.

Esta práctica es útil antes de hacer push al remoto, cuando aún es posible reescribir el historial local sin afectar a otros colaboradores.

Luego:

```
git add README.md  
git commit -m "docs: documentar correccion de historial en README"
```

FASE 3 — Ramas y resolución de conflicto

Rama 1: feature/visualizaciones

Paso 1 — Crear la rama:

```
git checkout -b feature/visualizaciones  
git branch
```

Paso 2 — Copia y ejecuta la sección **FASE 3 — Rama feature/visualizaciones** del notebook.

Paso 3 — Agrega al final del README.md:

- Rama visualizaciones: boxplot de colesterol agregado.

Paso 4 — Commits:

```
git add main.ipynb
git commit -m "feat: agregar boxplot de colesterol por condicion"

git add README.md
git commit -m "docs: registrar cambio en README rama visualizaciones"
```

Paso 5 — Volver a main:

```
git checkout main
```

Rama 2: feature/estadisticas

Paso 1 — Crear la rama desde main:

```
git checkout -b feature/estadisticas
```

Paso 2 — Copia y ejecuta la sección **FASE 3 — Rama feature/estadisticas** del notebook.

Paso 3 — Agrega al final del README.md (en la misma posición aproximada que la línea anterior — esto generará el conflicto):

- Rama estadisticas: resumen por grupo de edad agregado.

Paso 4 — Commits:

```
git add main.ipynb
git commit -m "feat: agregar resumen estadistico por grupo de edad"

git add README.md
git commit -m "docs: registrar cambio en README rama estadisticas"
```

Paso 5 — Volver a main:

```
git checkout main
```

Fusión de las dos ramas y resolución del conflicto

Paso 1 — Fusionar la primera rama (sin conflicto):

```
git merge feature/visualizaciones
```

Paso 2 — Fusionar la segunda rama (genera conflicto):

```
git merge feature/estadisticas
```

Git se detendrá indicando un conflicto en README.md.

Paso 3 — Entender el conflicto. Al abrir README.md verás:

```
<<<<<< HEAD
- Rama visualizaciones: boxplot de colesterol agregado.
=====
- Rama estadísticas: resumen por grupo de edad agregado.
>>>>>> feature/estadísticas
```

Paso 4 — Resolver el conflicto. Elimina las líneas con marcas («“<, =====, ””>») y deja ambas líneas de contenido:

```
- Rama visualizaciones: boxplot de colesterol agregado.
- Rama estadísticas: resumen por grupo de edad agregado.
```

Paso 5 — Confirmar la resolución:

```
git add README.md
git commit -m "merge: integrar visualizaciones y estadísticas en main"
```

Paso 6 — Verificar el historial final:

```
git log --oneline --graph
```

Copia el resultado en el README.md bajo la sección **## Historial final del proyecto** y haz:

```
git add README.md
git commit -m "docs: agregar historial final del proyecto en README"
```

FASE 4 — Publicación en GitHub (OPCIONAL)

Esta fase no es obligatoria para la evaluación.

1. Crea un repositorio vacío en <https://github.com> (sin README, sin .gitignore).
2. Genera un token de acceso personal en: Settings → Developer settings → Personal access tokens → Tokens (classic). Selecciona el scope **repo**.
3. Conecta el remoto:

```
git remote add origin https://github.com/tu_usuario/proyecto_cardio.git
```
4. Sube las ramas:

```
git push -u origin main
git push origin feature/visualizaciones
git push origin feature/estadísticas
```
5. Verifica en GitHub que el README, los commits y las tres ramas son visibles.

Preguntas Conceptuales

Marca con una **X** la única respuesta correcta.

Pregunta 1. ¿Qué ocurre exactamente cuando ejecutas `git add` sobre un archivo?

- a) El archivo se guarda en el repositorio con un nuevo commit.
- b) El archivo se copia al Staging Area, listo para el próximo commit.
- c) El archivo se sube automáticamente al repositorio remoto.
- d) Git crea una nueva rama con los cambios del archivo.

Respuesta: _____

Pregunta 2. Usaste `git reset HEAD~1` para deshacer un commit. ¿Cuál es la diferencia principal respecto a `git reset -hard HEAD~1`?

- a) Ambos comandos hacen exactamente lo mismo.
- b) Con `HEAD~1` se borra el commit pero los cambios en los archivos se conservan; con `-hard` se borra el commit Y los cambios también.
- c) Con `-hard` se crea una nueva rama; con `HEAD~1` no.
- d) `HEAD~1` borra el commit y `-hard` solo cambia el mensaje del commit.

Respuesta: _____

Pregunta 3. En la Fase 3, al fusionar las dos ramas a main, Git generó un conflicto en `README.md`. ¿Por qué ocurrió ese conflicto?

- a) Porque `main.ipynb` es un archivo binario que Git ignora siempre.
- b) Porque ambas ramas modificaron la misma zona del `README.md` y Git no puede decidir solo cuál versión conservar.
- c) Porque `README.md` no estaba incluido en el `.gitignore`.
- d) Porque Git solo genera conflictos en archivos de texto plano.

Respuesta: _____

Pregunta 4. ¿Qué muestra el flag `-graph` al ejecutar `git log --oneline --graph`?

- a) Un diagrama de los archivos modificados en cada commit.
- b) La representación visual de cómo se bifurcan y convergen las ramas en el historial de commits.
- c) La lista de colaboradores que participaron en cada commit.
- d) El diff de los cambios introducidos en cada commit.

Respuesta: _____

Pregunta 5. Un compañero revisa tu proyecto tres semanas después y ve este commit en el historial: `"feat: limpieza de valores invalidos y duplicados"`. ¿Qué ventaja concreta le da ese mensaje frente a uno como `cambios`?

- a) Ninguna. El mensaje de commit no afecta el funcionamiento del código.
- b) Le permite saber qué tipo de transformación se hizo y en qué parte del pipeline, sin necesidad de abrir el archivo ni el diff.
- c) Le permite revertir el commit más rápido usando `git revert`.
- d) Le indica en qué rama se hizo el commit originalmente.

Respuesta: _____

Tabla de Commits Esperados — Referencia Rápida

FASE 1:

chore: inicializar estructura del proyecto
chore: agregar notebook principal del pipeline
feat: cargar dataset cardiovascular desde UCI
feat: inspeccion inicial del dataset
feat: limpieza de valores invalidos y duplicados
feat: analisis exploratorio con tres visualizaciones
docs: registrar historial de fase 1 en README

FASE 2:

docs: agregar resultado clave del EDA en README
docs: documentar correccion de historial en README

FASE 3 (rama feature/visualizaciones):

feat: agregar boxplot de colesterol por condicion
docs: registrar cambio en README rama visualizaciones

FASE 3 (rama feature/estadisticas):

feat: agregar resumen estadistico por grupo de edad
docs: registrar cambio en README rama estadisticas

FASE 3 (merge en main):

merge: integrar visualizaciones y estadisticas en main
docs: agregar historial final del proyecto en README

Total: 16 commits.